Material de estudo

Predição de sepse neonatal tardia
com aprendizado de máquina

Conceitos por trás do pipeline do TCC

Este material reúne, em linguagem técnica mas não científica, os conceitos usados no pipeline: o que já foi feito, o que vem a seguir e por que cada etapa existe. Foi escrito para consulta e estudo ao longo de vários dias, não para leitura corrida.

Versão de 29 de outubro de 2026

Sumário

Como usar este material **3**

1 Visão geral: o problema e onde estamos **3**

1.1 O problema . . . 3

1.2 Os dados do projeto . . . 4

1.3 O processo KDD . . . 4

2 O princípio central: separar treino e teste e evitar vazamento **5**

2.1 Treino e teste . . . 5

2.2 Vazamento de dados (*data leakage*) . . . 5

3 O que já foi feito: pré-processamento e transformação **6**

3.1 Passo 1: carregar e auditar . . . 6

3.2 Passo 2: dividir . . . 6

3.3 Passo 3: o pré-processador . . . 6

3.3.1 Tipos de dados ausentes . . . 7

3.4 O Pipeline do scikit-learn . . . 7

3.5 O que está planejado mas foi adiado . . . 8

4 Validação: dobras, Pipeline, hiperparâmetros e GridSearch **9**

4.1 O problema que a validação resolve . . . 9

4.2 Dobras e validação cruzada . . . 9

4.3 Por que o Pipeline precisa ficar *dentro* da validação cruzada . . . 9

4.4 Parâmetros, hiperparâmetros e sobreajuste . . . 10

4.5 GridSearchCV . . . 10

5 Desbalanceamento de classes **11**

5.1 O problema . . . 11

5.2 `class_weight`: dar mais peso à classe rara . . . 12

5.3 SMOTE: fabricar casos sintéticos . . . 12

6 Métricas de avaliação **13**

6.1 A matriz de confusão . . . 13

6.2 As métricas . . . 13

6.3 Limiar, curva ROC e AUROC . . . 14

6.4 Calibração: a probabilidade é confiável? . . . 15

7 Os 8 algoritmos do Quadro 4 **19**

7.1 Descrição de cada um 19

7.2 Quadro comparativo 20

8 Interpretação: SHAP **20**

9 Como tudo se encaixa **22**

9.1 O fluxo completo 22

9.2 Mapa das etapas do repositório 22

9.3 Ordem proposta para a parte de modelos 23

10 Pontos em aberto para discutir com a orientadora **23**

A Glossário **23**

B Respostas do Teste-se **24**

Como usar este material

- Cada capítulo termina com uma caixa **Teste-se**. As respostas estão no Apêndice B.
- As caixas **azuis** definem conceitos, as **laranjas** apontam armadilhas e as **verdes** trazem exemplos numéricos.
- Os trechos de código são *esboços ilustrativos* para entender a ideia. Não são o código do repositório.
- O Apêndice A é um glossário com os termos em português e em inglês, porque a documentação das bibliotecas está em inglês.
- As métricas obtidas com os dados sintéticos **não têm valor científico**: o sinal foi plantado por nós. O que se valida agora é o *encanamento* do pipeline.

Ordem sugerida de leitura: capítulos 1 e 2 (visão geral e vazamento) → 3 (o que já foi feito) → 4 a 7 (a parte de modelos) → 8 a 10 (algoritmos, interpretação e encaixe final).

1 Visão geral: o problema e onde estamos

1.1 O problema

Queremos prever, a partir de dados do prontuário eletrônico (SOUL MV), se um recém-nascido internado na UTI neonatal vai ter **sepse neonatal tardia** (CID-10 A41.9 / P36). A ideia do aprendizado de máquina é mostrar ao programa muitos exemplos (bebês com seus dados e a informação de quem teve ou não sepse) e deixá-lo descobrir sozinho o padrão que liga uns aos outros. Depois, o padrão aprendido é usado em bebês novos.

Em termos técnicos, é um problema de **classificação binária supervisionada**:

- **Supervisionada**: cada exemplo vem com o *rótulo* (a resposta certa, aqui a coluna `sepse_tardia`).
- **Binária**: o rótulo só assume dois valores, sepse ou não.
- As colunas que descrevem o bebê são as **variáveis preditoras** (ou *features*), e a coluna a prever é o **alvo** (*target*).

Conceito: classificar × estimar uma probabilidade

"Binária" descreve o *rótulo* (sepse ou não), e é por isso que o problema é formalmente uma classificação binária. Mas isso **não** obriga o modelo a responder só "sim/não". O que o projeto quer entregar é uma **probabilidade de sepse tardia** (o risco do bebê, por exemplo 0,73). Na prática há duas camadas:

1. O modelo **estima a probabilidade** (nos algoritmos do scikit-learn, o método `predict_proba`).
2. Opcionalmente, um **limiar** (capítulo 6) transforma essa probabilidade em sim/não.

Os dois usos convivem, e as métricas do projeto (Quadro 5 do PDF) olham para camadas diferentes: a **AUROC** avalia a probabilidade (quão bem ela separa doentes de sadios); **F1, precisão, recall e acurácia** exigem a decisão sim/não, portanto dependem do limiar.

Atenção: calibração: a probabilidade é confiável?

Uma probabilidade é **calibrada** quando "30%" significa que cerca de 30% dos bebês com esse valor realmente têm sepse. AUROC e F1 *não* medem isso: um modelo pode ordenar bem os bebês e ainda assim dar probabilidades infladas ou deflacionadas. O texto do projeto cita a calibração como condição indispensável para a utilidade clínica (revisão da literatura, com Wong et al., 2021), mas as métricas do Quadro 5 não incluem nenhuma de calibração. É um ponto a levar à orientadora (seção 6.4 e capítulo 10). Dois cuidados práticos: algoritmos como árvore, KNN e Naive Bayes costumam dar probabilidades mal calibradas, e tanto o `class_weight` quanto o SMOTE **deslocam as probabilidades para cima** (o modelo "vê" mais sepse do que existe).

1.2 Os dados do projeto

Os dados reais dependem da aprovação do comitê de ética (CEP) e do conector com o prontuário. Enquanto isso, o pipeline é construído com **dados sintéticos** (o *seed*), gerados por `gerador_seed.py`:

- 1 linha por recém-nascido (tabela plana); no teste padrão, 1000 bebês.
- 99 colunas: 97 preditoras, mais `id_paciente` e `sepse_tardia`.
- Prevalência configurável (padrão 20% de sepse), sinal clínico plausível, exames seriados (`_pre` e `_pos24h`) e valores ausentes.

Atenção: o seed não é a realidade

Os pesos e efeitos do gerador são palpites clínicos. Os modelos vão "descobrir" o que nós mesmos plantamos. Com dados reais os efeitos serão mais complexos e o desempenho tende a ser pior.

1.3 O processo KDD

O projeto segue o **KDD** (*Knowledge Discovery in Databases*, Descoberta de Conhecimento em Bases de Dados), uma sequência de etapas. Vale reconhecer, em cada passo do código, em qual etapa você está.

Etapa do KDD	Status	O que acontece
1. Seleção	Feita	Escolha das variáveis (Quadros 2 e 3 do projeto) e geração do seed
2. Pré-processamento	Feito (passos 1–3)	Auditoria, divisão treino/teste, preenchimento de ausentes
3. Transformação	Feita (passo 3)	Escala, one-hot, `log1p`
4. Mineração de dados	**Próxima**	Treinar os 8 algoritmos do Quadro 4
5. Avaliação e interpretação	Depois	Métricas, curvas ROC, SHAP

Teste-se

1. O que significa dizer que o problema é de classificação binária supervisionada?
2. Qual a diferença entre variável preditora e alvo?
3. Por que as métricas do seed não servem como resultado científico?

2 O princípio central: separar treino e teste e evitar vazamento

2.1 Treino e teste

Um modelo que acerta os exemplos que já viu não prova nada: ele pode apenas ter decorado. O que importa é quão bem ele acerta bebês *que nunca viu*. Por isso os pacientes são divididos em dois grupos:

- **Treino (80%)**: onde o modelo aprende e onde todas as escolhas de desenvolvimento são feitas.
- **Teste (20%)**: uma prova final, **lacrada**. Só é aberta no fim, uma vez, para a nota honesta.

A divisão é feita **por paciente** (cada paciente cai em um só lado) e **estratificada** (os dois lados mantêm a mesma proporção de sepse), com uma semente fixa (`seed=42`) para ser reproduzível. No seed de 1000 bebês, ficam 800 no treino e 200 no teste.

2.2 Vazamento de dados (*data leakage*)

Conceito: vazamento de dados

É qualquer situação em que **informação que o modelo não teria no uso real** chega ao treino. O resultado é uma nota inflada, que desaba quando o modelo encontra dados novos. Há dois sabores importantes neste projeto:
(a) Vazamento do teste para o treino. Exemplo: calcular a mediana ou a escala usando todos os dados antes de dividir. Os números passam a carregar informação do teste. *Regra*: tudo o que é "aprendido" (medianas, escalas, pesos do modelo) vem **só do treino**.
(b) Vazamento do alvo. Uma variável que só existe, ou só é preenchida, *por causa* do desfecho. Ela entrega a resposta sem ter poder preditivo real.

Dois casos concretos já tratados neste projeto:

- `dias_ate_dx_sepse`: só existe para quem *teve* sepse (os dias até o diagnóstico). Quem não teve fica vazio. Logo ela prevê o alvo por construção, e foi **retirada** das variáveis de entrada.
- A **"zona cinzenta"**: `dias_utin`, `vm_dias`, `cvc_dias`, `npt_dias` e `hemocultura_*`. Um bebê com sepse tende a ficar mais dias internado, ventilado, com cateter etc., então essas variáveis podem ser *consequência* da sepse e não causa. Elas **ficam** como preditoras por enquanto, mas há um interruptor planejado para treinar sem elas (análise de sensibilidade). A recomendação para os dados reais é contar esses dias **até o momento da suspeita**, e não até a última evolução do prontuário.

Atenção: o vazamento mais traiçoeiro

O vazamento raramente dá erro: o código roda e as métricas aparecem lindas. Por isso a disciplina é de projeto (Pipeline, dobras, teste lacrado) e não de "conferir se ficou certo". Se uma métrica parece boa demais, desconfie primeiro do vazamento.

Teste-se

1. Por que não podemos usar o conjunto de teste para escolher entre dois modelos?
2. Por que `dias_ate_dx_sepse` foi removida, mas `dias_utin` ficou?
3. Calcular a mediana de uma coluna usando todo o dataset e depois dividir treino/teste é vazamento? Por quê?

3 O que já foi feito: pré-processamento e transformação

Os algoritmos esperam uma tabela numérica, completa e em escalas comparáveis. Os dados do prontuário não vêm assim. O módulo `preprocessamento.py` faz a ponte, em três passos concluídos.

3.1 Passo 1: carregar e auditar

Um "check-up" antes de qualquer coisa: quantas linhas, quantos ausentes por coluna, colunas quase constantes (95% ou mais com o mesmo valor, que não ajudam em nada) e colunas que sozinhas separam as classes quase perfeitamente (AUC acima de 0,95, forte suspeita de vazamento).

3.2 Passo 2: dividir

Separa 80% / 20%, como descrito no capítulo 2, e grava `data/treino.csv` e `data/teste.csv`.

3.3 Passo 3: o pré-processador

O pré-processador é construído a partir do treino e aplica transformações diferentes conforme o tipo de coluna. Conceitos usados:

Conceito: imputação pela mediana

Imputar é preencher valores ausentes. A **mediana** é o valor do meio quando se ordenam os números; ao contrário da média, não é puxada por valores extremos (um PCR de 300 não distorce o "valor típico"). A mediana usada é a **do treino**, inclusive para preencher os ausentes do teste.

Conceito: padronização (*standard scaling*)

Transforma cada coluna para ter média 0 e desvio-padrão 1:

$$z = \frac{x - \mu}{\sigma},$$

com μ e σ calculados **no treino**. Sem isso, uma coluna medida em milhares (plaquetas) dominaria outra medida em unidades (sexo 0/1) em qualquer algoritmo baseado em distância (KNN) ou em gradientes (regressão logística, MLP).

Conceito: one-hot encoding

Uma coluna de texto com categorias (por exemplo, três tipos de parto) é trocada por uma coluna 0/1 para cada categoria. O modelo não entende texto, e numerar as categorias (1, 2, 3) criaria uma ordem que não existe.

Conceito: `log1p`

Exames como PCR, procalcitonina, lactato e leucócitos têm **cauda longa**: muitos valores baixos e poucos muito altos. A transformação $\log(1+x)$ comprime os extremos, deixando a distribuição mais simétrica. O "+1" permite aplicar o log a zeros. É feita antes da padronização.

Conceito: indicador de ausência

Em dado clínico, *estar ausente* costuma carregar informação: um exame que ninguém pediu pode significar que o bebê parecia bem. Por isso, para exames com 20% ou mais de ausentes, o pré-processador cria uma coluna extra "foi medido?" (0/1), além de preencher o valor pela mediana. Isso preserva o sinal de que "faltou".

Caso especial, a hemocultura. O resultado vazio de uma cultura negativa não é "dado perdido": é "não se aplica". Esse vazio vira uma categoria própria, e não passa pela imputação.

3.3.1 Tipos de dados ausentes

- **MCAR** (*missing completely at random*): a ausência é puro acaso. É o caso do seed.
- **MNAR** (*missing not at random*): a ausência depende do próprio valor ou do estado do paciente. É o provável no dado real, e é por isso que existe o indicador de ausência.

3.4 O Pipeline do scikit-learn

Conceito: Pipeline e ColumnTransformer

Um **Pipeline** encadeia passos (imputar → escalar → modelo) em um único objeto, com as operações `fit` (aprender com os dados) e `transform`/`predict` (aplicar). Um **ColumnTransformer** aplica tratamentos diferentes a grupos de colunas (numéricas, binárias, categóricas) e junta o resultado.
A grande vantagem: o pipeline pode ser *refeito do zero* sobre qualquer subconjunto de dados. É por isso que ele é a base da validação cruzada sem vazamento (capítulo 4).

O resultado no seed: a matriz do treino passa a ter **800 linhas × 132 colunas, sem nenhum valor ausente** (as colunas preditoras originais viram 132 por causa do one-hot e dos indicadores de ausência).

3.5 O que está planejado mas foi adiado

O *passo 4* do pré-processamento oferece alternativas de imputação e um interruptor:

- **Imputação por KNN**: preenche o ausente com a média dos k pacientes mais parecidos (no treino).
- **Imputação iterativa**: faz regressões de cada coluna com ausentes sobre as demais, repetidamente. Mais lenta (cerca de 17 s por ajuste no seed, contra 0,3 s do KNN), o que a torna cara dentro de uma busca em grade.
- `excluir_sinalizadas`: treinar sem a zona cinzenta.

Decidiu-se priorizar os modelos e voltar a isso depois. A estratégia de imputação poderá virar mais um hiperparâmetro da busca.

Teste-se

1. Por que a mediana usada para preencher ausentes do teste deve vir do treino?
2. Qual a vantagem da mediana sobre a média para preencher ausentes de um exame de cauda longa?
3. Para que serve o indicador de ausência e em que tipo de ausência ele ajuda?
4. Por que padronizar é essencial para o KNN, mas pouco importante para uma árvore de decisão?

4 Validação: dobras, Pipeline, hiperparâmetros e GridSearch

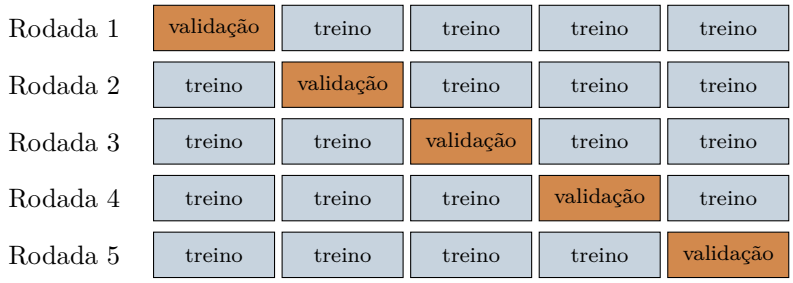
4.1 O problema que a validação resolve

Durante o desenvolvimento precisamos **comparar modelos e escolher configurações**, e não podemos usar o teste para isso (seria colar na prova final). Precisamos de uma "prova intermediária" feita só com o treino.

4.2 Dobras e validação cruzada

Conceito: validação cruzada de k dobras

Corta-se o conjunto de treino em k pedaços iguais, as **dobras** (*folds*). Em cada rodada, um pedaço serve de mini-prova (*validação*) e os outros $k-1$ servem de estudo. A nota final do modelo é a **média das k rodadas**. O projeto usa $k = 5$.



Por que isso ajuda:

- Compara modelos *sem tocar no teste*.
- Reduz o acaso. Com cerca de 100 a 175 casos de sepse, uma única divisão pode dar sorte ou azar. Cinco divisões dão uma nota mais estável.

Estratificada (`StratifiedKFold`) significa que cada dobra mantém a proporção de sepse do conjunto todo. Sem isso, uma dobra poderia ficar com poucos casos positivos, e a métrica daquela rodada ficaria instável.

4.3 Por que o Pipeline precisa ficar *dentro* da validação cruzada

Em cada rodada, o pipeline inteiro (mediana, escala, reamostragem, modelo) é **refeito do zero usando só as 4 dobras de estudo**. A dobra de validação nunca influencia a mediana, a escala ou o SMOTE daquela rodada. Se o pré-processamento fosse feito *antes* de cortar as dobras, a dobra de validação já teria contribuído para os números aprendidos, o que é vazamento (tipo a, do capítulo 2).

4.4 Parâmetros, hiperparâmetros e sobreajuste

Conceito: parâmetro × hiperparâmetro

Parâmetros são o que o modelo aprende sozinho dos dados (os pesos de uma regressão, as perguntas de uma árvore). **Hiperparâmetros** são os botões que *você* regula antes de treinar (número de vizinhos do KNN, profundidade máxima da árvore, taxa de aprendizado). Ninguém sabe de antemão o melhor valor.

Conceito: sobreajuste e subajuste

Sobreajuste (*overfitting*): o modelo decora o treino, incluindo o ruído, e vai mal em dados novos. **Subajuste** (*underfitting*): o modelo é simples demais e não capta nem o padrão real. Quase todo hiperparâmetro regula um ponto desse equilíbrio. Exemplo: uma árvore sem limite de profundidade decora o treino (sobreajuste); uma árvore com 1 pergunta só é simples demais (subajuste).

Atenção: risco específico do projeto

Com cerca de 700 pacientes, 15 a 25% de sepse (aproximadamente 105 a 175 casos) e 97 variáveis, o risco de sobreajuste é alto. O projeto não prevê seleção de variáveis; esse é um ponto a discutir com a orientadora.

4.5 GridSearchCV

Conceito: busca em grade

O **GridSearchCV** automatiza a escolha de hiperparâmetros:

1. Você define uma **grade**: uma lista de valores para cada hiperparâmetro.
2. Para **cada combinação**, ele executa a validação cruzada de 5 dobras e calcula a nota (aqui, o F1).
3. Escolhe a combinação com a melhor média.
4. **Retreina** essa combinação com todo o treino.

Exemplo: contas de custo

Grade do KNN: $k \in \{3, 5, 11, 21\}$ e `weights` $\in \{\text{uniform}, \text{distance}\}$: $4 \times 2 = 8$ combinações. Com 5 dobras, são $8 \times 5 = \mathbf{40}$ treinos, mais 1 retreino final. Se a grade tivesse 4 hiperparâmetros com 5 valores cada, seriam $5^4 = 625$ combinações $\times$ 5 dobras $= 3125$ treinos. O custo cresce multiplicativamente, e por isso se fala em grade *enxuta* (testar o pipeline) e *completa* (dado real).

```
# Esboço ilustrativo (não é o código do repositório)
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ("prep", preprocessador),          # imputar, escalar, one-hot
```

```
    ("modelo", KNeighborsClassifier()),
])
grade = {
    "modelo__n_neighbors": [3, 5, 11, 21],
    "modelo__weights": ["uniform", "distance"],
}
busca = GridSearchCV(
    pipe, grade,
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    scoring="f1",
)
busca.fit(X_treino, y_treino)           # 8 combinações x 5 dobras
print(busca.best_params_, busca.best_score_)
```

O prefixo `modelo__` (dois sublinhados) é a convenção do scikit-learn para "o hiperparâmetro `n_neighbors` do passo chamado `modelo`".

Atenção: a nota do vencedor é otimista

Como o vencedor foi escolhido *por ser o melhor naquelas dobras*, sua nota média de validação é um pouco otimista. A nota honesta vem do conjunto de teste lacrado. A versão mais rigorosa para estimar o desempenho de todo o procedimento é a **validação cruzada aninhada** (uma validação cruzada externa que contém o GridSearch interno); é mais cara e fica como opção.

Teste-se

1. Por que a nota de validação cruzada é mais confiável que uma única divisão treino/validação?
2. Qual a diferença entre parâmetro e hiperparâmetro? Dê um exemplo de cada.
3. Uma grade com 3 hiperparâmetros de 4 valores cada, com 5 dobras, faz quantos treinos?
4. Por que o pré-processamento tem que ficar dentro do Pipeline na validação cruzada?

5 Desbalanceamento de classes

5.1 O problema

Só cerca de 20% dos bebês têm sepse. Um modelo preguiçoso que responde "ninguém tem sepse" acerta 80% (acurácia de 80%) e **não serve para nada**: não acha nenhum caso. Por padrão, os algoritmos minimizam o erro total, e isso os leva a ignorar a classe pequena. Há duas famílias de solução.

5.2 `class_weight`: dar mais peso à classe rara

Conceito: pesos de classe

Não altera os dados. Informa ao modelo que errar um caso de sepse custa mais do que errar um caso sem sepse. No modo `balanced` do scikit-learn, o peso de cada classe é

$$w_c = \frac{n_{\text{amostras}}}{n_{\text{classes}} \cdot n_c}.$$

Com 20% de sepse: $w_{\text{sepse}} = 1/(2 \cdot 0{,}2) = 2{,}5$ e $w_{\text{sem sepse}} = 1/(2 \cdot 0{,}8) = 0{,}625$. Ou seja, um erro em sepse conta **4 vezes** mais.

Vantagens: simples, rápido, sem efeitos colaterais nos dados. Limitação: **nem todo algoritmo aceita** `class_weight`. No scikit-learn, `KNeighborsClassifier` e `MLPClassifier` não aceitam, e o `GradientBoostingClassifier` só aceita pesos por amostra (`sample_weight`) no `fit`. O XGBoost tem o equivalente `scale_pos_weight` ($\approx$ negativos/positivos $= 4$). Para esses casos, a alternativa é reamostrar.

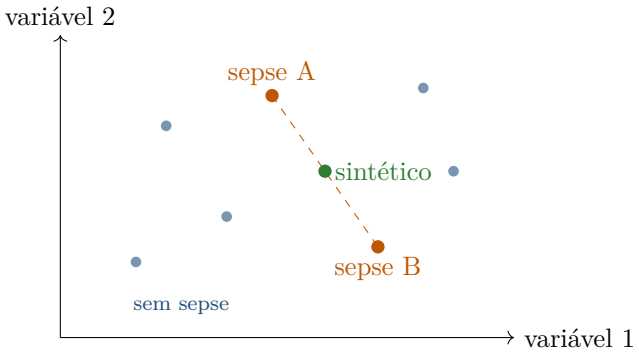
5.3 SMOTE: fabricar casos sintéticos

Conceito: SMOTE

Synthetic Minority Over-sampling Technique. Para cada novo caso sintético:

1. Sorteia um paciente **com sepse** do treino.
2. Acha seus k vizinhos mais próximos **também com sepse** (padrão $k = 5$).
3. Sorteia um deles e cria um ponto **no meio do caminho**: $x_{\text{novo}} = x + \lambda\,(x_{\text{vizinho}} - x)$, com λ sorteado entre 0 e 1.

Repete até a classe rara ficar com o tamanho desejado (por padrão, igualando as classes).



Limitações do SMOTE:

- **Só pode ser aplicado dentro do treino de cada rodada.** Se criasse sintéticos antes de dividir em dobras, cópias quase idênticas de um mesmo bebê cairiam no treino e na validação, e o modelo "reconheceria" o que já viu. É vazamento puro. Por isso ele fica *dentro* do Pipeline (na biblioteca `imbalanced-learn`, que tem um Pipeline próprio que só reamostra durante o `fit`).
- Misturar variáveis contínuas com 0/1 e one-hot gera pontos sem sentido (por exemplo, "sexo = 0,3"). Existe a variante SMOTE-NC para variáveis categóricas.

- Com cerca de 130 colunas e poucos casos, a noção de "vizinho" fica borrada.
- O teste e a validação **nunca** são reamostrados: a avaliação tem que refletir a prevalência real (20%).

Atenção

Não há resposta única sobre qual abordagem é melhor, `class_weight` ou SMOTE. O projeto prevê o SMOTE, e a ideia é comparar os dois com a mesma validação.

Teste-se

1. Por que uma acurácia de 80% pode ser inútil num problema com 20% de positivos?
2. O que o `class_weight='balanced'` faz com os dados? E o SMOTE?
3. Por que aplicar o SMOTE antes de dividir em dobras infla as métricas?
4. Cite um algoritmo do projeto que não aceita `class_weight` no scikit-learn.

6 Métricas de avaliação

6.1 A matriz de confusão

Para cada bebê, o modelo erra ou acerta de um de quatro jeitos:

	Previu sepse	**Previu sem sepse**
Teve sepse	Verdadeiro positivo (VP)	**Falso negativo (FN)**: caso que escapou
Não teve	Falso positivo (FP): alarme falso	Verdadeiro negativo (VN)

6.2 As métricas

Métrica	**Fórmula**	**Pergunta que responde**
Acurácia	$\frac{VP+VN}{VP+VN+FP+FN}$	Qual a fração de acertos? (enganosa com classes desbalanceadas)
Precisão	$\frac{VP}{VP+FP}$	Dos que o modelo marcou como sepse, quantos tinham mesmo? Posso confiar no alarme?
Recall (sensibilidade)	$\frac{VP}{VP+FN}$	Dos que tinham sepse, quantos o modelo achou? Quantos escaparam?
F1	$\frac{2 \cdot \text{precisão} \cdot \text{recall}}{\text{precisão} + \text{recall}}$	Média harmônica: só é alta se *ambas* forem altas

O **F1** é a métrica principal do projeto. A média *harmônica* é mais severa que a média comum: precisão 1,0 com recall 0,1 dá média comum de 0,55, mas F1 de apenas 0,18.

Em sepse neonatal o **falso negativo** costuma ser o erro mais grave (um bebê com sepse sem alarme), então o recall pesa clinicamente. Mas um recall alto à custa de muitos alarmes falsos sobrecarrega a equipe. O F1 equilibra os dois.

Exemplo: contas com 200 bebês de teste, 40 com sepse

Modelo A (acha 28 dos 40 casos; dá 20 alarmes falsos): VP = 28, FN = 12, FP = 20, VN = 140.

- Acurácia $= (28 + 140)/200 = 0{,}84$
- Precisão $= 28/(28 + 20) \approx 0{,}58$
- Recall $= 28/40 = 0{,}70$
- F1 $= 2 \cdot 0{,}58 \cdot 0{,}70/(0{,}58 + 0{,}70) \approx 0{,}64$

Modelo preguiçoso ("ninguém tem sepse"): VP = 0, FN = 40, FP = 0, VN = 160. Acurácia $= 160/200 = 0{,}80$ (parece decente!), mas recall = 0 e F1 = 0.

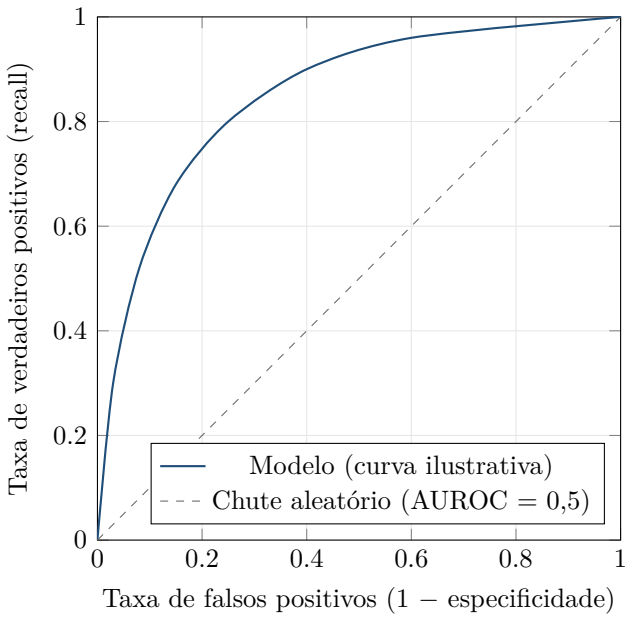
6.3 Limiar, curva ROC e AUROC

A maioria dos modelos não responde "sim/não": responde uma **probabilidade** ("73% de chance de sepse"). Para virar sim/não, escolhe-se um **limiar** (por exemplo, acima de 50% é sepse):

- **Limiar baixo**: pega mais casos (recall sobe), mas dispara mais alarmes falsos.
- **Limiar alto**: poucos alarmes falsos, mas deixa casos escaparem.

Conceito: curva ROC

Receiver Operating Characteristic. Para **todos os limiares de uma vez**, plota a taxa de acerto nos doentes (**taxa de verdadeiros positivos**, igual ao recall, eixo Y) contra a taxa de alarme falso entre os sadios (**taxa de falsos positivos** $= FP/(FP + VN)$, eixo X). Cada ponto da curva é um limiar. A diagonal é o chute aleatório; quanto mais colada no canto superior esquerdo, melhor.



Curva ilustrativa, não vem dos dados do projeto.

Conceito: AUROC

A área sob a curva ROC, de 0,5 (chute) a 1,0 (perfeito). Interpretação intuitiva: **é a probabilidade de, sorteando um bebê com sepse e um sem, o modelo dar nota mais alta ao doente**. Não depende de um limiar fixo.

Atenção: F1 e AUROC medem coisas diferentes

A **AUROC** avalia a capacidade de *ranquear* (separar doentes de sadios) em qualquer limiar. O **F1** avalia o desempenho *com um limiar específico* (em geral 0,5). Um modelo pode ter AUROC boa e F1 ruim simplesmente porque o limiar padrão não é adequado para uma classe rara. Ajustar o limiar (por exemplo, escolhendo o que maximiza o F1 dentro da validação cruzada) é uma decisão a considerar mais adiante. Com classes muito desbalanceadas, a curva *precisão* × *recall* também é informativa.

6.4 Calibração: a probabilidade é confiável?

Três números que se confundem

Número	Exemplo	O que é
Prevalência	20%	Proporção de bebês com sepse na *população*. É o ponto de partida: o que se chutaria sem saber nada sobre o bebê.
Probabilidade individual	"bebê X: 0,73"	Saída do modelo *para um bebê específico*, usando os dados dele. O modelo parte da prevalência e ajusta para cima ou para baixo.
Frequência observada	73%	Fração que *realmente* teve sepse entre os bebês que receberam probabilidades parecidas.

Conceito: calibração

Um modelo é **calibrado** quando a probabilidade que ele dá coincide com a frequência real: entre os bebês a quem ele atribui cerca de 0,73, cerca de 73% têm sepse. A probabilidade é **individual** (sobre o bebê X), mas só pode ser **verificada em grupo**: para um único bebê o desfecho é 0 ou 1 (teve ou não teve sepse), nunca "73%", então não há como dizer se o 0,73 estava "certo".

Discriminação e calibração são independentes

- **Discrimina bem, mas é mal calibrado:** o modelo dá 0,90 a todos os bebês com sepse e 0,80 a todos os sem sepse. A ordem é perfeita (AUROC = 1,0), mas ele diz "80% de risco" para bebês que nunca tiveram sepse.
- **Calibrado, mas inútil:** o modelo dá 0,20 a todo mundo. O número bate com a prevalência, mas não separa ninguém (AUROC = 0,5).

AUROC mede a discriminação. O F1 só olha a decisão sim/não depois do corte. Nenhum dos dois mede a calibração.

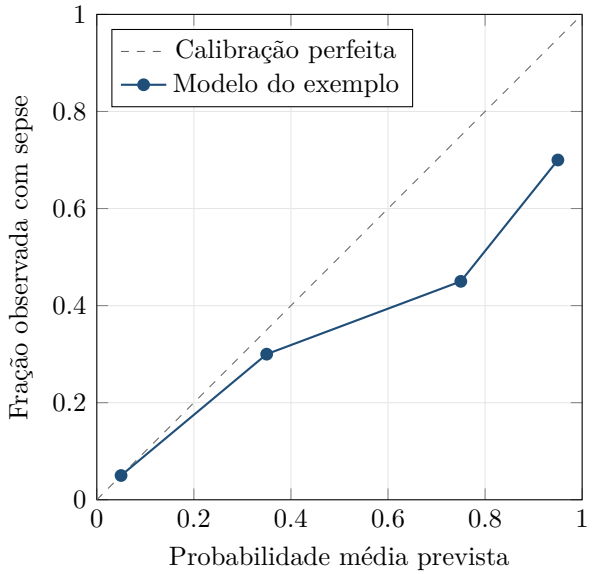
Como se mede

Curva de calibração (diagrama de confiabilidade):

1. Agrupe os bebês do teste por **faixa de probabilidade prevista** (0 a 10%, 10 a 20% etc.).
2. Em cada faixa, calcule a **média prevista** e a **fração que teve sepse**.
3. Plote uma contra a outra. A diagonal é a calibração perfeita. Pontos *abaixo* dela indicam que o modelo **superestima** o risco; pontos *acima*, que **subestima**.

Exemplo: um modelo bem calibrado nas faixas baixas e exagerado nas altas

Faixa prevista	Média prevista	Fração com sepse	Leitura
0 a 10%	5%	5%	calibrado
30 a 40%	35%	30%	quase
70 a 80%	75%	45%	**superestima**
90 a 100%	95%	70%	**superestima**



Curva do exemplo acima: os pontos abaixo da diagonal, à direita, mostram a superestimação.

Cada ponto compara, por faixa, o que o modelo previu com o que aconteceu. Não se trata de acertar bebê a bebê, e sim de a *média prevista* da faixa bater com a *frequência real*.

Escore de Brier: a média de $(p-y)^2$, com p a probabilidade prevista e y igual a 0 ou 1. Menor é melhor.

- Previu 0,9 e teve sepse: $(0{,}9-1)^2 = 0{,}01$ (erro pequeno). Previu 0,9 e não teve: $(0{,}9-0)^2 = 0{,}81$ (erro grande).
- **Referência ingênua:** quem prevê sempre a prevalência (0,20) tem Brier $= 0{,}2 \cdot 0{,}64 + 0{,}8 \cdot 0{,}04 = 0{,}16$, ou seja, prevalência $\times$ (1 − prevalência). Um modelo útil deve ficar **abaixo de 0,16**.
- O Brier mistura discriminação e calibração em um só número; por isso é usado junto com a curva.

Por que `class_weight` e SMOTE distorcem as probabilidades

No treino, os dois fazem o modelo "ver" a sepse como muito mais comum do que é (50% com SMOTE equilibrando; peso 4 vezes maior com `class_weight='balanced'`). O modelo continua acertando *quais* bebês são mais arriscados (a ordem se preserva, logo a AUROC não muda), mas parte de uma expectativa inicial errada. No caso idealizado, a correção é feita nas **odds** ($p/(1-p)$):

$$\text{odds reais} = \text{odds do modelo} \times \frac{\text{odds da prevalência real}}{\text{odds da prevalência vista no treino}}.$$

Com prevalência real de 20% (odds = 0,25) e treino equilibrado (odds = 1), multiplica-se por 0,25:

Modelo diz	Odds do modelo	Odds corrigidas ($\times 0{,}25$)	Probabilidade real
0,50	1	0,25	**0,20**
0,80	4	1,00	**0,50**
0,95	19	4,75	**0,83**

O modelo diz "80%" e o risco real daquele perfil está mais perto de 50%. A conta exata só vale no caso idealizado: com SMOTE, os pontos sintéticos distorcem ainda mais, então na prática a correção é feita *com dados*, por recalibração.

Isso também explica por que essas técnicas melhoram o recall e o F1 sem mudar a ordem dos bebês: usar `class_weight` e cortar em 0,5 equivale a usar o modelo "normal" com um corte bem mais baixo.

Como se corrige: recalibração

Aprende-se uma segunda função que leva a saída do modelo à frequência real, **em todas as faixas ao mesmo tempo**.

- **Platt scaling (sigmoide):** uma regressão logística de uma variável sobre a saída do modelo. Simples; funciona com poucos dados.
- **Regressão isotônica:** uma função em degraus, mais flexível, mas que exige mais dados.

Atenção: a recalibração precisa de dados *não vistos*

A função de recalibração deve ser ajustada em dados que o modelo **não usou para treinar** (por exemplo, as dobras de validação); caso contrário, volta o vazamento. A avaliação final da calibração é feita no **teste**, que mantém a prevalência real. No scikit-learn, o `CalibratedClassifierCV` faz isso dentro da validação cruzada.

```python
# Esboço ilustrativo (não é o código do repositório)
from sklearn.calibration import calibration_curve, CalibratedClassifierCV
from sklearn.metrics import brier_score_loss

p = modelo.predict_proba(X_teste)[:, 1]            # probabilidade de sepse
freq_real, media_prev = calibration_curve(y_teste, p, n_bins=5, strategy="quantile")
print(brier_score_loss(y_teste, p))                # ingênuo com 20%: 0,16

# recalibrar com dados não usados no treino (sigmoide)
calibrado = CalibratedClassifierCV(modelo, method="sigmoid", cv=5)
```

Calibração × limiar: duas coisas diferentes

	Calibração	**Limiar (corte)**
Pergunta	Os números do modelo batem com a realidade em todas as faixas?	A partir de qual probabilidade disparo o alarme?
Natureza	Corrigir **todos** os números (uma função aplicada à curva inteira)	Escolher **um** valor
Como	Platt scaling ou regressão isotônica	Por exemplo, maximizar o F1 na validação
"Ponto ótimo"	Não há: o objetivo é que todas as faixas fiquem na diagonal	Há: o corte que melhor atende ao critério escolhido

As duas se combinam: primeiro se corrigem os números (calibração), depois se escolhe o corte conforme o uso clínico. Com números calibrados, o corte é mais fácil de justificar (por exemplo, "alarme quando o risco real estimado passar de 30%").

Tendências por algoritmo e limites práticos

- **Regressão logística:** costuma sair razoavelmente calibrada, se não houver reponderação forte.
- **Naive Bayes:** tende a ser exagerado, empurrando valores para perto de 0 e 1.
- **Random Forest:** costuma ser conservador, com valores puxados para o meio.
- **Árvore e KNN:** dão valores grosseiros (0; 0,2; 0,4...), porque dependem de poucas folhas ou poucos vizinhos.
- **Poucos casos:** com 105 a 175 casos de sepse, a curva fica ruidosa. Use poucas faixas (cerca de 5) e prefira a sigmoide à isotônica.
- **Quando importa mais:** se o produto final é um alarme sim/não, a calibração pesa menos. Se o médico vai ver "risco de 73%", ela é essencial: probabilidades infladas geram excesso de alarmes e fadiga de alarmes, que é o problema citado no projeto sobre o modelo avaliado por Wong et al. (2021).

Teste-se

1. Um modelo tem VP = 18, FN = 22, FP = 12, VN = 148. Calcule acurácia, precisão, recall e F1.
2. Por que o falso negativo é, em geral, o erro mais grave neste problema?
3. O que acontece com recall e alarmes falsos quando se *abaixa* o limiar?
4. Interprete uma AUROC de 0,82 com as suas palavras.
5. O modelo dá 0,80 ao bebê X. O que significa dizer que ele está calibrado nessa faixa? Dá para verificar isso olhando só o bebê X?
6. Por que recalibrar por sigmoide não muda a AUROC?
7. Um modelo treinado com classes equilibradas (50%) diz 0,80 para um perfil, e a prevalência real é de 20%. Qual a probabilidade real aproximada?
8. Qual a diferença entre calibrar as probabilidades e escolher o limiar?

7 Os 8 algoritmos do Quadro 4

Cada algoritmo é uma "família" diferente de aprender. Como ninguém sabe qual funciona melhor com o dado real, o projeto compara oito abordagens.

7.1 Descrição de cada um

Regressão Logística (RL) Soma ponderada das variáveis, transformada em probabilidade por uma curva em S (função logística). Cada peso diz quanto a variável aumenta ou diminui o risco, o que a torna muito interpretável. O botão principal é a **regularização** (`C`): penaliza pesos grandes para evitar sobreajuste.

Naive Bayes (NB) Aplica o teorema de Bayes **supondo que as variáveis são independentes entre si** (por isso "ingênuo"). Rapidíssimo e bom como linha de base, mas essa suposição raramente vale na prática (por exemplo, PCR e procalcitonina andam juntas).

KNN (k vizinhos mais próximos) Para classificar um bebê novo, procura os k bebês **mais parecidos** no treino e vota. Não há treino de verdade: ele guarda os dados e consulta na hora. "Parecido" é uma *distância*, então as variáveis precisam estar na mesma escala. Com muitas colunas a distância perde sentido (*maldição da dimensionalidade*: em muitas dimensões, todos os pontos ficam quase igualmente distantes). O botão principal é o k: pequeno demais vira ruído (sobreajuste); grande demais vira uma média sem graça (subajuste).

Árvore de Decisão (DT) Sequência de perguntas sim/não ("PCR $>$ 12? plaquetas $<$ 100?"). Muito interpretável, mas se aprofunda demais e decora o treino. Botões: `max_depth`, `min_samples_leaf`.

Random Forest (RF) Muitas árvores, cada uma treinada numa amostra aleatória dos pacientes e das colunas, que depois **votam**. A diversidade entre as árvores compensa o defeito de cada uma (reduz variância). Resistente ao sobreajuste e pouco sensível a escala.

Gradient Boosting (GB) Árvores **em sequência**: cada nova árvore tenta corrigir os erros das anteriores. Botões: `n_estimators`, `learning_rate` (passo de cada correção) e `max_depth`. Costuma ser muito forte em dados tabulares, mas é sensível aos botões.

XGBoost Uma implementação muito otimizada e regularizada do gradient boosting. Geralmente a favorita em dados tabulares. Tem mais botões (`subsample`, `colsample_bytree`, `scale_pos_weight`).

MLP (rede neural) Camadas de "neurônios" que combinam as entradas e aplicam funções não lineares. Flexível, mas sensível aos botões e à escala, precisa de mais dados e é o menos interpretável. Botões: `hidden_layer_sizes`, `alpha` (regularização), `learning_rate_init`.

Atenção: dois "KNN" diferentes

O KNN que **classifica** (este capítulo) não é o KNN que **preenche ausentes** (passo 4 do pré-processamento). A ideia dos vizinhos é a mesma; o uso, outro. O SMOTE também usa vizinhos, de um terceiro jeito.

7.2 Quadro comparativo

Algoritmo	Exige escala?	Interpretável?	`class_weight`?	Botões típicos
Reg. Logística	Sim	Alta	Sim	`C`, penalidade
Naive Bayes	Não	Média	Não (usa priors)	`var_smoothing`
KNN	**Sim**	Média	**Não**	`n_neighbors`, `weights`
Árvore (DT)	Não	Alta	Sim	`max_depth`, `min_samples_leaf`
Random Forest	Não	Média	Sim	`n_estimators`, `max_depth`, `max_features`
Gradient Boosting	Não	Média/baixa	Só `sample_weight`	`n_estimators`, `learning_rate`, `max_depth`
XGBoost	Não	Média/baixa	`scale_pos_weight`	idem, mais `subsample`
MLP	**Sim**	Baixa	**Não**	`hidden_layer_sizes`, `alpha`

Como o pré-processador já padroniza tudo, os modelos que exigem escala estão atendidos, e os demais não são prejudicados.

Teste-se

1. Por que o KNN depende de as variáveis estarem na mesma escala?
2. Qual a diferença de ideia entre Random Forest e Gradient Boosting?
3. Que sintoma você esperaria de uma árvore sem limite de profundidade?
4. Por que o Naive Bayes é chamado de "ingênuo"?

8 Interpretação: SHAP

Depois de escolher o melhor modelo, a pergunta passa a ser *por que* ele prevê o que prevê.

Conceito: valores SHAP

O **SHAP** (*SHapley Additive exPlanations*) atribui a cada variável, **em cada paciente**, o quanto ela empurrou a previsão para cima ou para baixo em relação a uma previsão média. Vem dos valores de Shapley da teoria dos jogos: pensa-se nas variáveis como jogadores que cooperam para produzir a previsão e divide-se o "crédito" de forma justa.

Usos típicos: um gráfico de importância global (quais variáveis mais pesam no modelo) e explicações individuais (por que *este* bebê recebeu risco alto). Para o contexto clínico é essencial, porque um modelo preciso mas opaco dificilmente é aceito.

Atenção

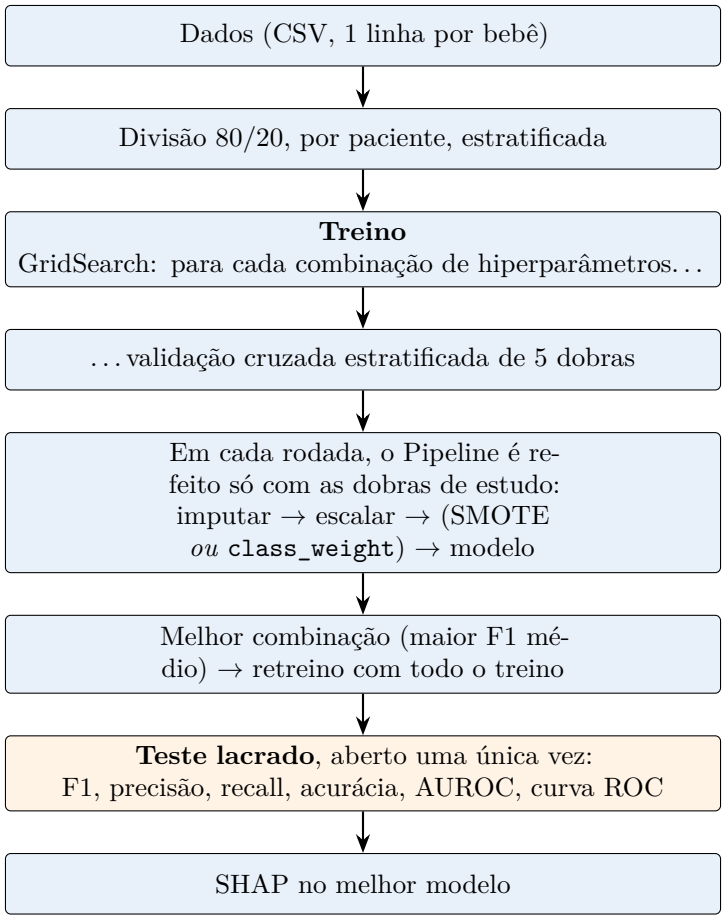
SHAP explica o **modelo**, não a **realidade**. Se o modelo usa uma variável que é consequência da sepse (zona cinzenta), o SHAP vai mostrá-la como importante, e isso é um alerta para o desenho dos dados, não uma descoberta clínica.

Teste-se

1. O que um valor SHAP positivo para “PCR” num paciente significa?
2. Por que uma importância SHAP alta para `dias_utin` deve ser interpretada com cuidado?

9 Como tudo se encaixa

9.1 O fluxo completo



Esse fluxo roda para cada um dos 8 algoritmos, e o resultado final é a comparação entre eles.

9.2 Mapa das etapas do repositório

Etapa	Status	Conteúdo
0. Dados sintéticos	Concluída	`gerador_seed.py`
1. Pré-processamento	Passos 1–3 feitos	`preprocessamento.py`; passo 4 (KNN/iterativo, interruptor) adiado
2. Desbalanceamento	Pendente	`class_weight` e SMOTE dentro da validação cruzada
3. Mineração de dados	Pendente	8 algoritmos + GridSearchCV
4. Avaliação	Pendente	Métricas, validação cruzada $k = 5$, curvas ROC; calibração (proposta, a validar com a orientadora)
5. Interpretação	Pendente	SHAP

9.3 Ordem proposta para a parte de modelos

1. Módulo de modelos com os 8 algoritmos e suas grades (enxuta e completa, escolhidas por uma opção), treino com `class_weight` e validação cruzada. Já dá um pipeline de ponta a ponta.
2. SMOTE como alternativa e o interruptor `excluir_sinalizadas`.
3. Avaliação no teste e curvas ROC.
4. SHAP e, se necessário, os refinamentos do passo 4.

10 Pontos em aberto para discutir com a orientadora

- **Zona cinzenta.** Contar `dias_utin`, `vm_dias`, `cvc_dias` e `npt_dias` **até o momento da suspeita**, e não até a última evolução. Isso depende do conector com o prontuário.
- **Seleção de variáveis.** Com cerca de 97 preditoras e 105 a 175 casos de sepse, o risco de sobreajuste é alto, e o projeto não prevê seleção de variáveis.
- **Colapso das colunas `_pre` e `_pos24h`.** Todas as 99 colunas foram mantidas; juntar as janelas fica para depois, se necessário.
- **Limiar de decisão.** O padrão de 0,5 pode ser inadequado para uma classe rara; ajustar o limiar é uma decisão de desenho.
- **Calibração das probabilidades.** Se o produto final é o risco do bebê, convém acrescentar ao Quadro 5 uma curva de calibração e o escore de Brier; `class_weight` e SMOTE distorcem as probabilidades, o que pode exigir recalibrar.
- **Procedimento de imputação.** O PDF do projeto não define qual usar (a referência da p. 40 aponta para um texto que não existe). A mediana é o padrão atual.

A Glossário

Termo (PT)	Termo (EN)	Definição curta
Variável preditora	feature	Coluna que descreve o paciente e entra no modelo
Alvo	target / label	Coluna a prever (`sepse_tardia`)
Treino / teste	train / test	Dados para aprender / prova final lacrada
Vazamento	leakage	Informação indevida chegando ao treino, inflando a nota
Dobra	fold	Um dos k pedaços da validação cruzada
Validação cruzada	cross-validation	Rodar k mini-provas, trocando a dobra de validação
Estratificada	stratified	Mantém a proporção das classes em cada divisão
Hiperparâmetro	hyperparameter	Botão regulado antes do treino
Busca em grade	grid search	Testar todas as combinações de uma grade de hiperparâmetros
Sobreajuste	overfitting	Decorar o treino e falhar em dados novos
Subajuste	underfitting	Modelo simples demais para o padrão
Imputação	imputation	Preencher valores ausentes

Termo (PT)	Termo (EN)	Definição curta
Padronização	standardization	Deixar média 0 e desvio-padrão 1
One-hot	one-hot encoding	Categoria vira colunas 0/1
Desbalanceamento	class imbalance	Classes com tamanhos muito diferentes
Pesos de classe	class weights	Erro na classe rara pesa mais
SMOTE	SMOTE	Criação de casos sintéticos da classe rara
Matriz de confusão	confusion matrix	Tabela de VP, FP, VN, FN
Precisão	precision	VP / (VP + FP)
Recall / sensibilidade	recall / sensitivity	VP / (VP + FN)
F1	F1-score	Média harmônica de precisão e recall
Limiar	threshold	Corte que transforma probabilidade em sim/não
Calibração	calibration	Grau em que a probabilidade prevista bate com a frequência real ("30%" $\approx$ 30% de casos)
Escore de Brier	Brier score	Média de (probabilidade − resultado)^2, com resultado 0 ou 1; menor é melhor
Curva ROC	ROC curve	Taxa de verdadeiros positivos × falsos positivos, para todos os limiares
AUROC	AUC / AUROC	Área sob a curva ROC
Pipeline	pipeline	Encadeamento de passos tratado como um único objeto
SHAP	SHAP	Contribuição de cada variável para cada previsão
KDD	KDD	Processo de descoberta de conhecimento em bases de dados
MCAR / MNAR	MCAR / MNAR	Ausência aleatória / ausência que depende do valor ou do estado

B Respostas do Teste-se

Capítulo 1. (1) Há exemplos com resposta conhecida (supervisionada) e o rótulo só tem dois valores (binária). O modelo, porém, estima uma probabilidade, que opcionalmente vira sim/não por um limiar. (2) Preditora descreve o paciente e entra no modelo; alvo é o que se quer prever. (3) O sinal foi plantado pelo gerador; os modelos "descobrem" o que nós colocamos, e o dado real será mais difícil.

Capítulo 2. (1) Seria escolher o modelo olhando a prova final; a nota deixaria de ser honesta. (2) A primeira só existe para quem teve sepse (vazamento comprovado). A segunda pode ser consequência da sepse, mas também é plausível como preditora; é uma suspeita, e por isso fica com um interruptor de sensibilidade. (3) Sim: a mediana passa a carregar informação das linhas que irão para o teste.

Capítulo 3. (1) Para o teste ser tratado como dado novo e não influenciar o que foi aprendido. (2) A mediana não é puxada pelos valores extremos da cauda longa. (3) Preserva o sinal de que o exame não foi medido; ajuda quando a ausência é informativa (MNAR). (4) O KNN usa distâncias, e uma coluna em escala grande dominaria; a árvore só compara cada coluna com um corte, então a escala não muda as divisões.

Capítulo 4. (1) Faz a média de várias divisões, reduzindo o efeito do acaso, sobretudo com poucos casos positivos. (2) Parâmetro é aprendido (peso da regressão); hiperparâmetro é regulado

por você (número de vizinhos). (3) $4 \times 4 \times 4 = 64$ combinações $\times\, 5 = 320$ treinos (mais o retreino final). (4) Para que as estatísticas (mediana, escala) sejam recalculadas só com as dobras de estudo de cada rodada.

Capítulo 5. (1) Um modelo que nunca prevê sepse já tem 80%, mas não acha nenhum caso. (2) `class_weight` não altera os dados, só o peso do erro; o SMOTE cria casos sintéticos da classe rara. (3) Cópias quase iguais de um caso ficam no treino e na validação; o modelo é avaliado em algo que já viu. (4) KNN (e também o MLP).

Capítulo 6. (1) Acurácia $= (18 + 148)/200 = 0{,}83$; precisão $= 18/30 = 0{,}60$; recall $= 18/40 = 0{,}45$; F1 $= 2 \cdot 0{,}60 \cdot 0{,}45/1{,}05 \approx 0{,}51$. (2) Um bebê com sepse sem alarme pode ter o tratamento atrasado. (3) O recall sobe e os alarmes falsos também. (4) Em 82% dos pares (um com sepse, um sem), o modelo dá nota maior ao bebê com sepse. (5) Que, entre os bebês a quem ele atribui cerca de 0,80, cerca de 80% têm sepse. Não: para um bebê só o desfecho é 0 ou 1; a verificação é em grupo. (6) A sigmoide é uma transformação crescente: preserva a ordem dos bebês, e a AUROC só depende da ordem. (A isotônica pode criar empates e alterar um pouco.) (7) Odds do modelo $= 4$; multiplicadas por 0,25, dão 1; logo, probabilidade de cerca de 0,50. (8) Calibrar corrige todos os números, em todas as faixas, para que batam com a frequência real; o limiar é um único valor que decide quando disparar o alarme.

Capítulo 7. (1) Porque "parecido" é medido por distância; colunas com escala grande dominariam o cálculo. (2) Na RF as árvores são independentes e votam; no GB cada árvore corrige os erros das anteriores. (3) Sobreajuste: ótimo no treino, bem pior em dados novos. (4) Porque supõe que as variáveis são independentes entre si, o que raramente é verdade.

Capítulo 8. (1) Que, para aquele paciente, o valor do PCR empurrou a previsão para mais risco de sepse, em relação à média. (2) Porque pode ser consequência da sepse (zona cinzenta); a importância alta pode refletir vazamento temporal e não efeito causal.